RISC-V Based SoC Design Featuring Fixed and Floating Point Units with Integrated FFT Accelerator for DSP Workloads

Umer Kudachi
School of Electronics and Communication Engineering
KLE Technological University, Hubballi, India
mrumeryk@gmail.com

Saroja V. Siddamal
Department of Electrical and Electronics Engineering
KLE Technological University, Hubballi, India
sarojavs@kletech.ac.in

Shreevatsa Alawandi
School of Electronics and Communication Engineering
KLE Technological University, Hubballi, India
shreevatsaalawandi@gmail.com

Aishwarya Beedanal
School of Electronics and Communication Engineering
KLE Technological University, Hubballi, India
aishwaryabeedanal2@gmail.com

***Abstract*—The proposed work aims to design and implement a 32-bit in-order RISC-V processor, implemented on the Artix-7 (Arty 7) FPGA board. The design follows a 5-stage pipelined architecture—Fetch, Decode, Execute, Memory, and Writeback—with an optimized hazard detection unit that handles data hazards effectively. A Harvard architecture is employed to reduce structural hazards and enhance instruction throughput. The processor supports IEEE 754-compliant single-precision floating-point arithmetic, enabling accurate handling of real-number computations essential for scientific and signal processing tasks. To mitigate control hazards, a branch prediction unit has been implemented, achieving up to 40% efficiency in worst-case scenarios. The RISC-V processor has been transformed into a system-on-chip (SoC) by integrating a Serial Peripheral Interface (SPI) module, which enables communication with external slave devices through data transmission and reception. Instruction encoding is optimized using the RARS Simulator to accelerate the conversion from assembly to machine code. In order to extend the processor's capability beyond general-purpose computing, a dedicated FFT co-processor based on the Cooley-Tukey algorithm has been integrated, enabling efficient acceleration of compute-intensive DSP operations such as spectral analysis and signal filtering. The design is developed using the Vivado Design Suite 2024.2, achieving a maximum operating clock frequency of 48 MHz with an on-chip power consumption of 108 mW. Experimental validation confirms functional correctness and performance efficiency. The paper details the processor architecture, design methodology, hardware integration, and performance evaluation, positioning it against comparable RISC-V implementations for embedded and signal processing workloads.**

***Index Terms*—RISC-V SoC Design, Pipelined Architecture , Hardware Accelerator, Co-processor Integration**

I. Introduction

The open-source RISC-V Instruction Set Architecture (ISA) has emerged as a highly flexible and modular alternative to traditional proprietary ISAs, making it ideal for academic, research, and industrial applications. Due to its low complexity, ease of implementation, and support for customization, the RISC-V ISA has become popular in both general-purpose and domain-specific processor designs [1]. The RISC-V ISA was initiated in May 2010 at UC Berkeley by Prof. Krste Asanović and his students, including Yunsup Lee and Andrew Waterman, as part of the Parallel Computing Laboratory. It was designed as a clean-slate, open-source ISA to support academic and industrial innovation. The first specification was released in 2011, with the first chip taped out the same year. The RISC-V Foundation (now RISC-V International) was launched in 2015 to drive global adoption [2]. A variety of RISC-V-based open-source processors have emerged over the years, including BOOM (Berkeley Out of Order Machine), SiFive's E300 and U500 platforms, and the Shakti series developed by IIT Madras. The mentioned processors feature advanced pipelined architectures to boost performance while maintaining low hardware costs. However, due to their architectural complexity, they can pose challenges for beginners and learners attempting to grasp the fundamental concepts of the RISC-V instruction set. In order to bridge the complexity gap found in existing RISC-V designs, the proposed work

introduces a simplified 32-bit RISC-V processor core tailored for educational and entry-level design purposes. The processor is built with an emphasis on modularity and straightforward architecture, making it accessible to those new to processor design. The architecture of the processor is shown in Figure 1, and the block diagram of the SoC is presented in Figure 2.

II. Literature Review

Several works have explored the design and implementation of RISC-V processors with varying goals and architectures. Rajyan and Saini [3] present a modular, single-cycle RV32I processor core emphasizing manual instruction decoding and simulation-driven verification for educational use.

Poli et al. [4] design a minimal, single-cycle RV64I processor with a non-pipelined Harvard architecture, focusing on educational simplicity and low resource usage. Their design achieves 2.276 DMIPS/MHz, making it suitable for compact educational deployments. Tiwari et al. [5] introduce IndiRA, a pipelined RV32I processor with power and area efficiency optimizations, including a dynamic branch predictor and reduced critical path delay. This design improves performance by 30% and is verified through FPGA and ASIC flows.

Singh et al. [6] implement a folded six-stage pipelined RV32I processor with a unique Pre-Decode stage for compressed instruction expansion and early branching. This design reduces stalls and improves throughput, validated on high-performance FPGAs.

Nair et al. [7] focus on simplicity and modularity with a 5-stage pipelined RV32IM processor, implemented on an Artix-7 Basys 3 FPGA, featuring static branch prediction and a custom UART module, making it suitable for educational purposes and real-time data verification.

While many designs focus on modular pipelining and basic ISA implementations, none integrate a complete SoC with advanced features for DSP applications. Our design stands out by incorporating a dedicated FFT co-processor, dynamic branch prediction, hazard management, and an IEEE 754-compliant floating-point unit, ideal for DSP tasks. Additionally, the inclusion of SPI protocol support expands its capability for real-time communication in embedded systems.

III. Methodology

The section outlines the structural and functional blueprint of the processor. The processor was built using AMD's Vivado Design Suite version 2024.2 and follows a classic five-stage pipeline approach, as illustrated in Fig. 1. The pipeline includes the following sequential stages: (i) Fetching the instruction, (ii) Decoding the fetched instruction, (iii) Executing the operation, (iv) Accessing memory , and (v) Writing the result back to the register file. The processor maintains an in-order execution model across all these stages. The detailed explanation of the processor's top-level architecture and its internal pipeline components is presented in the upcoming subsections.

A. Fetch Unit

The first stage of the processor pipeline is the fetch stage, where the primary function is to retrieve the next instruction to be executed. At the beginning of this stage, the Program Counter (PC) is incremented to point to the memory address of the upcoming instruction. This updated PC value is then used to access the instruction memory, allowing the corresponding instruction to be fetched. Once retrieved, the instruction is stored in the IF-ID pipeline register, which holds it temporarily for the next decode stage. Instruction memory is directly mapped to the PC, ensuring that the correct instruction is consistently accessed in every cycle.

To enhance the efficiency of this stage, a specialized module known as the Bimodal Branch Prediction Unit is incorporated within the Fetch Unit. This unit is designed to minimize stalls associated with branch instructions. Typically, whether a branch is taken or not is determined only during the execution stage, resulting in up to two stall cycles for regular branch operations. The Branch Protection Unit mitigates these delays using a Finite State Machine (FSM)-based logic. Each FSM operates as a four-state transition predictor, following the standard two-bit branch prediction scheme. The four states include: strongly not taken (00), weakly not taken (01), weakly taken (10), and strongly taken (11). These states transition based on the actual outcomes of previous branches, dynamically updating predictions. For instance, if a branch is frequently taken, the FSM remains or moves toward the strongly taken state; otherwise, it transitions toward the strongly not taken state. This predictive mechanism allows the fetch stage to make early decisions on branch outcomes, significantly reducing unnecessary stalls and improving instruction throughput. The integration of FSM-based prediction logic ensures a more intelligent and adaptive instruction flow, contributing to the overall performance and responsiveness of the processor pipeline. The efficiency of the bimodal branch predictor which is modeled using Markov Chains, denoted as $\eta(p)$, is defined as the correct prediction rate, i.e.,

$$\eta(p) = \frac{1 - 2p + 2p^2}{-4p^4 + 8p^3 - 6p^2 + 2p + 1} \tag{1}$$

The expression quantifies predictor accuracy as a function of branch bias p, aiding architectural optimization under varying control-flow behaviors. The worst-case scenario occurs when $p = 0.5$, resulting in an efficiency of approximately 40%. The best-case scenario occurs when $p = 0$ or $p = 1$ (or close to either), where the efficiency reaches 100%.

B. Decode Stage

The Decode Unit is responsible for interpreting the fetched instruction and generating the necessary control signals and data for execution. It comprises several key components. The Sign Extension block extends immediate values from instruction formats like I-type and S-type (12-bit to 32-bit) and J-type (26-bit to 32-bit), ensuring compatibility with the processor's 32-bit data path. The Control Unit, which is responsible for generating control signals is divided into

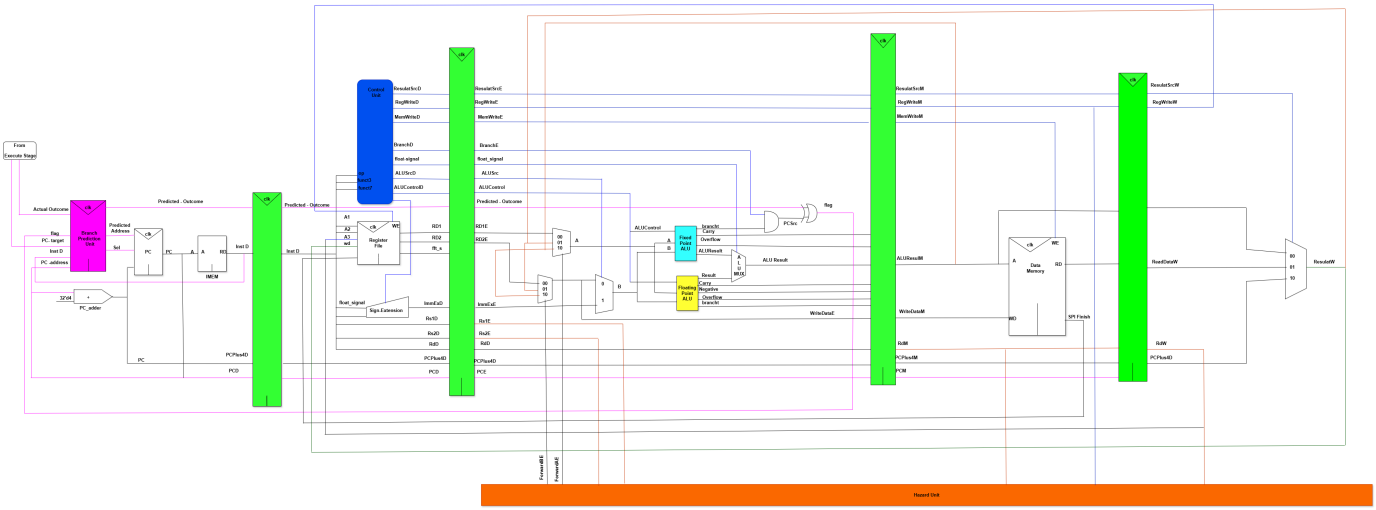


Fig. 1: Architecture of the Proposed RISC V Core. Readers are advised to zoom in to observe finer details.

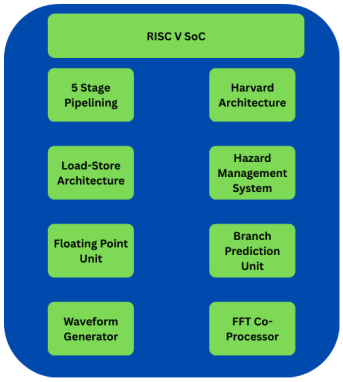


Fig. 2: Block Diagram of the SoC

TABLE I: Supported Instructions in the RISC-V Processor

Category	Instruction Type	Instructions
Integer Arithmetic	R-type	ADD, SUB, MUL, DIV
	I-type	ADDI
Memory Operations	I-type	LW
	S-type	SW
Branching	B-type	BNE, BEQ, BLT, BGE
Shifting	R-type	SLL, SRL, SRA
Comparison	R-type	SLT, SLTU
Bitwise Operations	R-type	AND, OR, XOR
Floating Point Load/Store	I-type / S-type	FLW, FSW
Floating Point Arithmetic	R-type	FADD.S, FSUB.S, FMUL.S, FDIV.S

two parts: the Main Decoder, which interprets the opcode to determine actions like register/memory write, branching, and ALU or memory operations, and the ALU Decoder, which specifies the exact ALU operation (e.g., addition, subtraction, AND, OR). The Register File reads data from the source registers (RS1, RS2) used in ALU operations and stores the result during the Write Back stage. Lastly, the ID/EX Register serves as a pipeline buffer between the Decode and Execute stages, forwarding control signals, register values, and the sign-extended immediate value to ensure stable and synchronized execution in the next stage.

C. Execute Unit

The execution unit forms a vital stage of the processor pipeline and contains the Arithmetic Logic Unit (ALU), which handles all arithmetic, logical, and branch computations. The decode unit generates specific control signals based on the instruction, and these signals guide the ALU to perform the intended operation. For example, if the decode stage outputs the control signal for addition, then the ALU executes only the addition operation; the same logic applies for subtraction, multiplication, and others. Within the execution cycle, multiplexers (MUXes) determine whether to select the immediate value or the RS2 register value as the second operand for the ALU. This selection enables support for both immediate and register-based instructions. The flags which are essential for getting the correct PC address for the branch prediction unit are calculated in the execute stage.

D. Floating Point Module

The floating point module designed in this work is a modular and IEEE-754 single-precision compliant arithmetic logic unit (ALU) that performs fundamental floating-point operations such as addition, subtraction, multiplication, and division. A key advantage of this module lies in its implementation of division using the Newton-Raphson iterative method, which accelerates the computation of reciprocals and enhances overall efficiency compared to traditional non-iterative techniques. Furthermore, the design ensures correct rounding behavior and precision as per the IEEE-754 standard, including support for special cases such as NaN (Not-a-Number), infinities, and subnormals. Compared to conventional monolithic designs, this modular architecture offers better maintainability, scalability, and adaptability for integration into larger floating-point

processing pipelines or custom processors. The RTL schematic of the floating point ALU unit is given the figure 3. Overall, the module provides a reliable and reusable foundation for high-performance floating-point computation in hardware design.

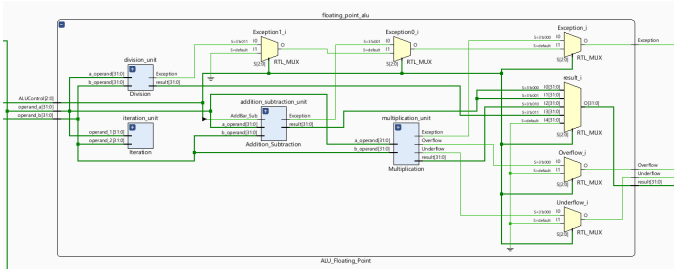


Fig. 3: RTL Schematic of the Floating Point ALU

E. Memory Access and Writeback

The Memory Access and Writeback stages are vital for managing data flow between the processor and memory. In the Memory Access (MEM) stage, the ALU result from the Execute stage is used either as a memory address (for load/store instructions) or forwarded as a final computation result. Load instructions (e.g., LW) fetch data from memory, while store instructions (e.g., SW) write data—decoded earlier—from the register file into memory. This stage also interacts with the Hazard Unit to handle potential hazards, especially those caused by load-use dependencies. The Writeback (WB) stage is the final stage, responsible for writing results to the register file. A multiplexer selects between the memory-loaded data (for load instructions) and the ALU result (for arithmetic operations) to be written back. For store instructions, memory writes are already completed in the MEM stage. Additionally, the Writeback stage provides updated results to the Hazard Unit to support data forwarding and resolve instruction dependencies as illustrated in Fig 1.

F. Hazard Management System

To resolve data hazards arising from read-after-write dependencies in a pipelined processor, a hazard detection unit dynamically forwards the most recent data to the execution stage. If a source register in the execution stage matches the destination register of an instruction in the memory or writeback stage, and the corresponding instruction intends to write to the register file, the updated value is forwarded from the respective later stage. This ensures that the execution stage receives the correct operand without stalling the pipeline. The following diagram illustrates a specific scenario where such data hazards occur and are resolved through operand forwarding. One of the worst-case scenario, shown in figure 4, occurs when both source registers in the second instruction depend on the destination register of the first. Furthermore, the third instruction has data dependencies on both the first and second instructions. Figure 5 refers to how the hazard unit interacts with the ALU.

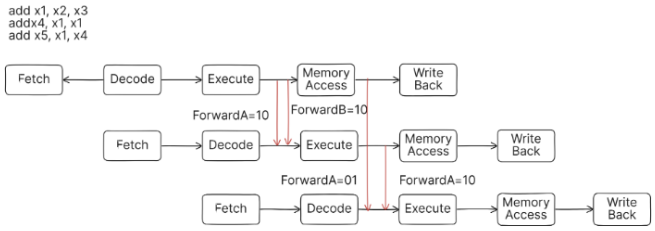


Fig. 4: Worst Case Scenario

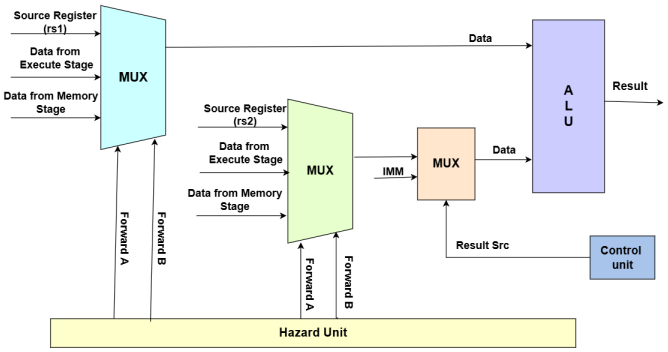


Fig. 5: Hazard Unit Interaction with the ALU

G. SPI Communication Module

The SPI module is memory-mapped to address `0x01` in the data memory, enabling the processor to initiate a transmission by writing to this location. The design focuses on master-side data transmission using SPI Mode 0 (CPOL = 0, CPHA = 0). A Finite State Machine (FSM) with five distinct states—`IDLE`, `LOAD`, `TRANSMIT`, `DONE`, and `RESET`—controls the data flow. In the `IDLE` state, the module waits for a valid write signal from the processor. Upon detection, it moves to the `LOAD` state, where the 8-bit data is latched into a shift register and the slave select (SS) line is pulled low. The `TRANSMIT` state serially shifts data out through the MOSI line on every rising edge of the SPI clock, which is internally divided from the system clock. A 3-bit counter tracks the number of bits sent. After all 8 bits are transmitted, the FSM transitions to the `DONE` state, where the SS line is deasserted, and a finish flag is raised to signal transmission completion. The processor detects this and writes the value `1` to register `x30`, indicating success. Finally, the FSM enters the `RESET` state to clear internal signals and return to `IDLE` for the next operation.

It is important to note that the FFT coprocessor and SPI module operate independently, while both are controlled and coordinated by the main RISC-V processor. This modular architecture allows concurrent or sequential execution of FFT computations and SPI data transfers without interference, ensuring flexibility and efficient resource utilization within the system. The FSM-based approach ensures synchronized, bit-accurate transmission and seamless integration into the pipelined RISC-V environment.

TABLE II: SPI Timing and Data Rate Parameters

Parameter	Value
Processor Clock Frequency (f_{clock})	48 MHz
Clock Cycles per SPI Bit (clk_per_bit)	2
SPI Clock Frequency (f_{SPI})	12 MHz
Time per SPI Clock Cycle (T_{SPI})	83.33 ns
Time per Byte (8 bits)	666.66 ns
Data Rate	12 Mbps (1.5 MBps)

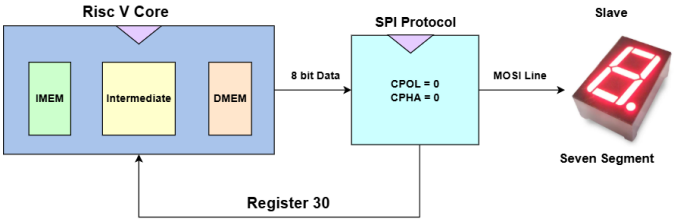


Fig. 6: Interaction between RISC-V Core, SPI and a Slave

H. FFT Co-Processor

In the designed RISC-V SoC, an FFT coprocessor is integrated to efficiently execute 8-point Decimation in Time (DIT) Fast Fourier Transform (FFT) operations. The computation follows a three-stage pipeline, where each stage handles specific butterfly operations. Stage 0 processes 2-point butterflies, Stage 1 performs 4-point FFTs, and Stage 2 completes the full 8-point transformation. Each stage includes dedicated hardwired butterfly units that operate sequentially on the input data. To avoid data hazards and maintain orderly computation, each stage only proceeds after the previous stage has completed. A Finite State Machine (FSM) governs the transitions between stages, and no data moves forward unless the receiving stage is ready, effectively managing backpressure and maintaining correct sequencing.

The main processor and FFT coprocessor execute in parallel while accessing a shared memory space. Memory-mapped architecture enables both units to interact with the same physical memory. To manage access without conflict, an arbitration mechanism assigns memory locks—one indicating the memory is occupied by the CPU and the other indicating it is in use by the FFT coprocessor. Only one unit can access memory at any given moment.

When the processor issues a memory instruction, such as load or store, it checks whether the memory is currently in use by the FFT coprocessor. If so, the processor halts in the decode stage and holds the program counter to avoid incorrect instruction execution. Once the FFT coprocessor releases the memory, the processor locks it for its own use, performs the memory operation, and then releases it, allowing further access by the FFT coprocessor.

Similarly, before accessing memory, the FFT coprocessor checks whether the processor has completed its memory transaction. If the memory is still locked by the processor, the coprocessor waits. Once the processor releases control, the coprocessor locks the memory for its own use. The coprocessor remains idle until it receives a trigger from the processor indicating that an FFT operation should begin. Once active, it reads input data from memory, processes it across the three pipeline stages, and prepares the result. If the required memory locations for writing results are not yet ready, the coprocessor remains in a waiting state. Once those locations are available, the FFT results are written back to memory, and memory control is relinquished. At that point, the processor is notified that the FFT operation has completed, and memory is again available for general use.

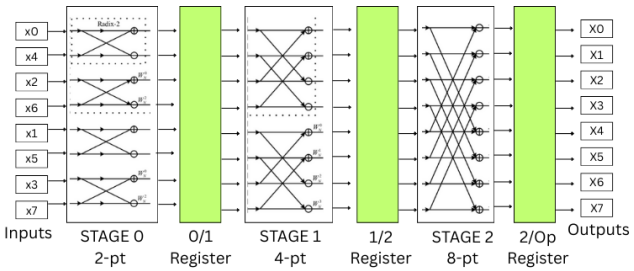


Fig. 7: Micro-architecture of the FFT Unit

Coordinated use of memory locks and readiness flags ensures that the processor and coprocessor function in parallel without interfering with each other. Arbitration logic prevents simultaneous memory access, ensuring data consistency and system stability. The decode stage in the processor plays a crucial role in reducing unnecessary stalls. If a memory instruction is encountered while the coprocessor holds memory access, the processor temporarily stalls until the coprocessor releases it. Likewise, the coprocessor waits until the processor completes its transaction before initiating access. This controlled sequencing avoids contention and minimizes processing delays.

The architecture combines pipelined FFT computation with synchronized memory arbitration to achieve efficient parallelism. Processor and coprocessor operate independently, yet remain tightly coordinated when interacting with shared memory. This balance of concurrency and control improves overall system performance, making the design highly suitable for real-time embedded systems that demand reliable and concurrent processing with minimal latency. The design achieves high parallel efficiency by combining FSM-based stage control with memory arbitration logic, enabling concurrent processing while safeguarding data integrity. By coordinating access privileges and managing pipeline dependencies, the architecture supports simultaneous execution of processor and coprocessor tasks, enhancing throughput and ensuring reliable operation for memory-intensive embedded applications. The micro-architecture of the FFT co-processor is given in the figure 7.

IV. Testing of FFT Coprocessor

A real audio file (`.wav`) was used as the input signal for evaluating the FFT coprocessor. Using Python libraries such as `NumPy` and `Matplotlib`, eight representative time-domain samples were extracted from the audio data. These

same samples, generated from the software environment, were also provided as inputs to the hardware FFT coprocessor to ensure a direct and fair comparison.

Frequency-domain outputs were obtained from both the software-based FFT implementation and the hardware coprocessor. The results from the two approaches were compared, and the error between them was calculated and plotted. Figures 8 and 9 show the software-generated inputs/outputs and the corresponding processor-side signals, respectively, while Fig. 10 presents the error plot. The time-domain input values are transmitted to the processor, with all the eight values taking eight clock cycles to be written into memory. Once stored, the FFT computation begins independently, starting from the seventeenth memory location. The results are organized in an interleaved format, where the seventeenth location contains the real part of the first FFT output, the eighteenth location contains the corresponding imaginary part, and this pattern continues sequentially for all subsequent outputs. As observed in the error plot in figure 10, the maximum error is 0.002, which is negligible given the immense complexity of the FFT unit. This memory layout ensures efficient storage and retrieval of FFT results for further processing. The diagrams remain consistent with the earlier stages of experimentation, confirming the validity of the results.

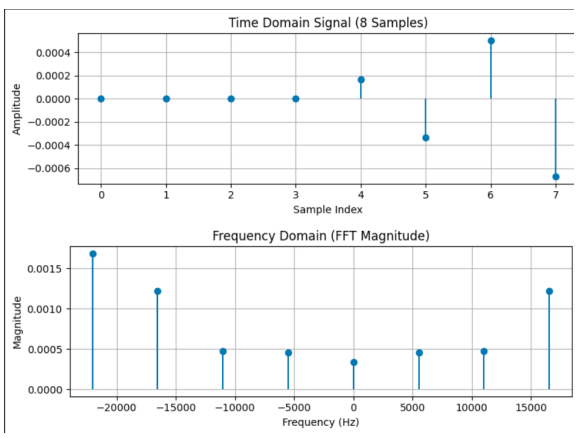


Fig. 8: Software-based FFT results showing both time-domain samples and frequency-domain output.

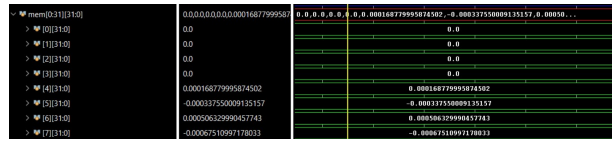
V. Simulation Results and Analysis

Figure 11 shows the computation of circle area using $A = \pi r^2$, where $r = 2.3$ (stored in `Register 7`) and $\pi = 3.142$ (in `Register 6`). The value r^2 is stored in `Register 9`, and the final result πr^2 in `Register 10`.

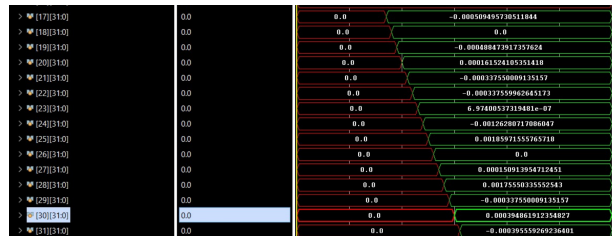
SPI communication protocol usage is demonstrated in Figure 12, where the character 'A' is displayed on the seven-segment module from the processor. Up counters and down counters have also been designed.

A. Maximum Clock Frequency

The maximum frequency (F_{max}) is calculated using the formula:



(a) Time-domain input to the processor.



(b) Frequency-domain output from the processor.

Fig. 9: Processor-side (a) time-domain input and (b) frequency-domain output signals.

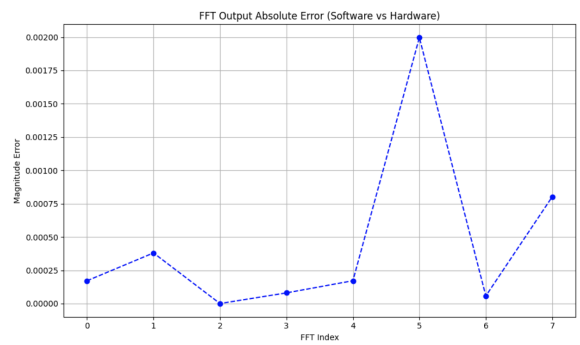


Fig. 10: Error plot between software-based FFT results and processor-based FFT results.

$$F_{\text{max}} = max(\frac{1000}{T_i - \text{WNS}}) \tag{2}$$

The maximum frequency of the system is 48 MHz without the floating-point division module. For a lightweight RISC-V model with complex computational units such as the floating-point unit (which handles operations like addition, subtraction, and multiplication), a maximum frequency of 48 MHz is quite reasonable. However, when the floating-point division module is included, the maximum frequency drops to around 7 MHz. This reduction is due to the increased resource usage and computational complexity introduced by the division operation.

B. Power Consumption Report

Table III presents the estimated power consumption of the synthesized RISC-V processor, obtained using Vivado's power analysis tool. The processor consumes a total of **108 mW**, with a majority of the power being static.

Key Observations:

- The processor exhibits low power consumption, making it suitable for energy-constrained and embedded applications.

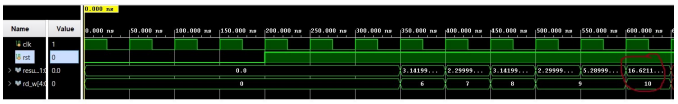


Fig. 11: Simulation of Area of Circle

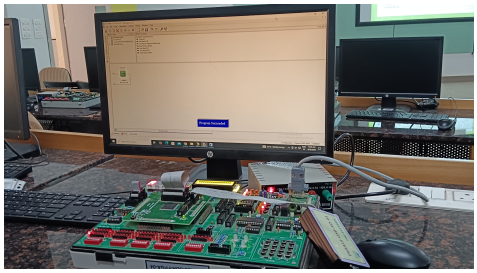


Fig. 12: Experimental setup showing communication with a slave; 'A' displayed on the seven-segment display via the processor

- Static power accounts for 84% of the total power, indicating the dominance of leakage current, which is typical in FPGA-based implementations.
- Logic contributes the highest share of dynamic power (45%), reflecting core computational activity.
- Signals consume 37% of dynamic power, suggesting moderate interconnect usage and switching.
- The junction temperature remains low at 25.5 °C, well within safe thermal margins.

C. Resource Utilization

The FPGA resource utilization of the synthesized 32-bit pipelined RISC-V processor was analyzed post-synthesis using Xilinx Vivado on the Artix-7 (xc7a100t) FPGA.

It is important to note that the FFT and SPI modules are memory-mapped peripherals that operate independently of the main RISC-V pipeline. These are included in the total resource utilization for completeness but are not considered in core-specific evaluations. All optimization, performance, and area analyses for the RISC-V processor exclude these peripheral modules.

TABLE III: Power Consumption Report of the Synthesized Processor

Parameter	Value
Total On-Chip Power	0.108 W
Device Static Power	0.091 W (84%)
Dynamic Power	0.017 W (16%)
– Clocks	0.001 W (8%)
– Signals	0.006 W (37%)
– Logic	0.008 W (45%)
– DSP	0.001 W (6%)
– I/O	0.001 W (4%)
Hierarchical Power	0.017 W
Junction Temperature	25.5 °C
Thermal Margin	74.5 °C (15.9 W)
Ambient Temperature	25.0 °C
Effective θJA	4.6 °C/W

Name	Slice LUTs	Slice Registers	F7 Muxes	F8 Muxes	DSPs	Bonded IOB	BUFGCTRL
TOP MODULE	26281	6080	689	118	37	34	4
Fetch Cycle	160	134	3	0	0	0	0
Decode Cycle	1098	1117	192	64	0	0	0
executecycle	8584	106	224	27	21	0	0
memorycycle	16461	4723	270	27	16	0	0

Fig. 13: FPGA Resource Utilisation of RISC-V Processor Modules

The total LUT usage of the top module without including the FFT and SPI models is 5799 LUTs

Key Observations

- The execute-cycle module accounts for a significant share of LUT usage, indicating high arithmetic and control logic complexity.
- The Floating point ALU alone consumes 18 DSPs, highlighting its dependence on dedicated arithmetic resources.
- The memory-cycle is a major consumer of logic because the FFT unit and SPI module are mapped into it.
- The register file utilizes a high number of flip-flops (992), aligning with its function as a central storage element.
- The Branch Prediction Unit is resource-light, making it an efficient addition to the pipeline for performance gain.
- The SPI module is extremely lightweight and has negligible effect on overall resource utilization.

VI. Competitive Landscape

TABLE IV: Core Overview of Compared Processors

Core / Processor	Source	Architecture Type
PicoRV32	Open-source	RISC-V
SiFive E21 Coreplex	Commercial	RISC-V
ARM Cortex-M4	Commercial	ARMv7-M
ESP32	Commercial	Xtensa-based
Proposed	Open/Custom	RISC-V

TABLE V: Feature Comparison with Competitors

Feature	Proposed	PicoRV32	SiFive E21	Cortex-M4	ESP32
RISC-V ISA	✓	✓	✓	×	×
5-Stage Pipeline	✓	×	Partial	✓	✓
Hazard Management	✓	×	✓	✓	✓
Floating Point Unit	✓	×	×	✓	✓
FFT Co-processor	✓	×	×	×	×
Harvard Architecture	✓	×	✓	✓	✓
SPI Module	✓	✓	✓	✓	✓
Customizability	High	High	Medium	Low	Low
Open Source	✓	✓	×	×	×

The proposed work stands out by providing a balanced mix of high-performance execution and architectural flexibility. Figure below shows how the proposed processor compares to popular alternatives in terms of performance and customizability.

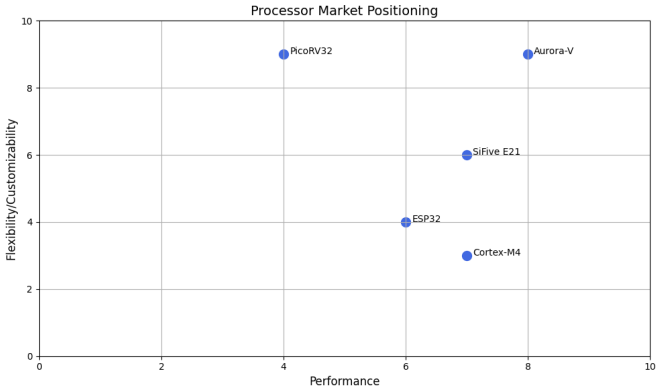


Fig. 14: Market Positioning: Flexibility vs. Performance

VII. Conclusion and Future Scope

The development of a RISC-V processor with a focus on floating-point operations and an integrated FFT coprocessor marks a significant step forward in addressing the needs of modern digital signal processing (DSP) applications. The design demonstrates the potential of RISC-V architecture to provide both flexibility and high-performance computing in specialized tasks like FFT, essential for a wide range of applications, from audio and video processing to scientific computations. By integrating a dynamic branch prediction unit, hazard management system, and support for a wide array of instructions, the processor ensures improved efficiency and performance. Furthermore, the inclusion of an SPI protocol interface and the development of a comprehensive System-on-Chip (SoC) design paves the way for future advancements in embedded systems, IoT, and real-time DSP applications. Future research may focus on optimizing floating-point units, memory management, and low-power design strategies. Moreover, incorporating machine learning coprocessors and enhancing security features can expand the processor's capabilities, ensuring that RISC-V continues to play a pivotal role in next-generation computing systems. The work serves as a foundation for further innovations in processor design, offering valuable insights and new directions for future research in the RISC-V ecosystem.

References

[1] K. Asanović and D. A. Patterson, "Instruction Sets Should Be Free: The Case for RISC-V," EECS Department, University of California, Berkeley, Tech. Rep. UCB/EECS-2014-146, Aug. 2014.

[2] RISC-V International, "About RISC-V – History," *RISC-V International*. [Online]. Available: https://riscv.org/about/history. [Accessed: Apr. 7, 2025].

[3] A. Rajyan, G. Saini, et al., "SystemVerilog based design of an RV32I compliant RISC-V processor core," in *Proc. 2024 5th IEEE Global Conf. for Advancement in Technology (GCAT)*, Karnataka, India, Oct. 2024, pp. 1–6, doi: 10.1109/GCAT62922.2024.10923874.

[4] L. Poli, S. Saha, K. D. Mcdonald-Maier, and X. Zhai, "Design and implementation of a RISC V processor on FPGA," in *Proc. 2021 17th Int. Conf. on Mobility, Sensing and Networking (MSN)*, Essex, UK, Dec. 2021, pp. 161–166, doi: 10.1109/MSN53354.2021.00037.

[5] A. Tiwari, P. Guha, G. Trivedi, N. Gupta, N. Jayaraj, and J. Pidanic, "IndiRA: Design and Implementation of a Pipelined RISC-V Processor," 2023 33rd International Conference Radioelektronika (RADIOELEKTRONIKA), Pardubice, Czech Republic, Apr. 2023, pp. 1–6, doi: 10.1109/RADIOELEKTRONIKA57919.2023.10109058.

[6] A. Singh, N. Franklin, N. Gaur, and P. Bhulania, "Design and Implementation of a 32-bit ISA RISC-V Processor Core using Virtex-7 and Virtex-UltraScale," 2020 5th International Conference on Computing Communication and Automation (ICCCA), Greater Noida, India, Oct. 2020, pp. 126–129, doi: 10.1109/ICCCA49541.2020.9250816.

[7] P. Nair and L. V., "Design and Implementation of 5-Stage Pipelined RISC-V Processor on FPGA," 2024 28th International Symposium on VLSI Design and Test (VDAT), Trivandrum, India, July 2024, pp. 1–6, doi: 10.1109/VDAT63601.2024.10705665.